Exercise 08: SAT PyUVM randomization

Objective: *Introduction to constrained randomization PyUVM testbench using the PyVSC library.*

Task:

The files for this exercise are in `<ROOT>/lecture04/exercises/E07_sat_pyuvm_uvc`. This directory contains the `Makefile` and the testbench files. In the following sections, file names are relative to `tb/`.

Updates to the testbench setup

In Exercise 07, the *Saturation Filter* testbench you implemented a default test and a default virtual sequence. This exercise introduces a base test `sat_tb_base_test.py` and a base virtual sequence `sat_tb_base_seq.py`. They hold the setup shared by the tests and sequences, without specifying the stimuli for an individual test. The `sat_tb_base_test` also does not start the virtual sequence on the virtual sequencer. The default test and default sequence have been updated accordingly. Inspect the source files and look at the implementation of the default test to understand the setup.

Testcases now reside in `tests/` and their respective virtual sequences in `vseqs/`. The default test and default sequence have been moved there already.

Note: Imports in these directories should be absolute as seen from the `tb/` directory. For example in a test in `tests/example_test.py`

```
# Import the base test
from sat_tb_base_test import sat_tb_base_test

# Import a virtual sequence from tb/vseqs
from vseqs.example_seq import example_seq
```

Your editor might not like these imports and complain, but the Makefile used to run the test will execute the test with `tb/` as the root directory.

Logging

Additionally a new command line flag is introduced: `PYUVM_LOG_LEVEL`. This flag is resolved in the class constructor of `sat_tb_base_test` and controls the loglevel of *PyUVM*. All components in *PyUVM* have their own logger and can log messages at the typical levels (`DEBUG`, `CRITICAL`, `ERROR`, `WARNING`, `INFO`, `NOTSET`, `NullHandler`).

```
# In any pyuvm component:
self.logger.debug("This is a debug message.")
self.logger.info("This is a info message.")
```

Part 1: Constraints on Stimuli

First we will create four random testcases:

- A fully random testcase sending random `data` values.
- A testcase sending only values below the `THRESHOLD` (64) of the *Saturation Filter* filter.
- A testcase sending only values above the `THRESHOLD` of the *Saturation Filter* filter.
- A testcase sending only in a band around the `THRESHOLD` of the *Saturation Filter* filter.

Random Stimuli

In `vseqs/sat_tb_random_seq.py`:

- Create `sat_tb_random_seq` extending `sat_tb_base_seq`.
- Mirror the implementation of the `sat_tb_default_seq` but instead of setting the `data` value of the sequence item directly randomize it.

```
seq_item = uvc_ssdt_seq_item.create(name="producer_item")
seq_item.randomize()
self.producer_seq.seq_item = seq_item
```

- Send 10 transactions with random data though the DUT.

In `tests/sat_tb_random_test.py` create a new testcase which extends `sat_tb_base_test` and overrides the `sat_tb_base_seq` with `sat_tb_random_seq`. Hint: You can take a look at `tests/sat_tb_default_test.py` as a reference.

Constrained Random Stimuli

Next, we constrain the randomization of the sequence item. Instead of calling `seq_item.randomize()` add a constraint with for example:

```
with seq_item.randomize_with() as item:
    item.data < 200
```

Create three virtual sequences using the setup as `sat_tb_random_seq`.

- In `vseqs/sat_tb_less_than_threshold_seq.py` implement a sequence that constrains the data to only values below the threshold.
- In `vseqs/sat_tb_over_threshold_seq.py`, implement a sequence that constrains the data to only values above the threshold.
- In `vseqs/sat_tb_around_threshold_seq.py`, implement a sequence that constrains the data to only values in a band around the threshold (E.g. `THRESHOLD` $\pm$ 10)

Create three respective testcases in `tests/`. Each testcase extends `sat_tb_base_test`, overrides `sat_tb_base_seq` with its corresponding virtual sequence, and starts that sequence in `run_phase` as shown above.

Select the new test modules in `COCOTB_TEST_MODULES` in the `Makefile` and run them.

Part 2: Constraints on Sequence Number

Next, we randomize the number of *sSDT* transactions sent by a virtual sequence.

In `vseqs/sat_tb_rnd_n_seq.py`:

- Create a new virtual sequence and decorate it with `@vsc.randobj`.
- In the constructor, create producer and consumer `uvc_ssdt_default_seq` instances and a `vsc.rand_uint32_t()`.

```
self.number_of_seqs = vsc.rand_uint32_t()
```

- Add a constraint which constrains `number_of_seqs` $0 < n \leq 100$

```
@vsc.constraint
def number_of_seqs_pos_max(self):
    self.number_of_seqs > 0
    self.number_of_seqs <= 100
```

- In the `body` run `number_of_seqs` sequences, with fully random data values.

In `tests/sat_tb_rnd_n_seq_test.py` create a new test, which launches the sequence like any of the other tests, but before `<seq>.start()` call `<seq>.randomize()`.

Lastly add another test in which you add an additional constraint on the minimum `number_of_seqs` the `sat_tb_rnd_n_seq` using `randomize_with`.

```
with self.virtual_sequence.randomize_with() as seq:
    seq.number_of_seqs >= 10
```

Part 3: Analyzing Distribution

Use the producer responses to inspect the distribution of the generated input values.

- In the virtual sequences from Parts 1 and 2, initialize a `producer_responses` list. After each producer transaction, append `int(self.producer_seq.rsp.data)`. The producer response contains the driven input value, not the DUT output.

```
await Combine(prod_task, cons_task)
self.producer_responses.append(int(self.producer_seq.rsp.data))
```

- In the corresponding tests in `tests/`, import `render_histogram` from `sat_tb_histogram.py`.
- After the virtual sequence completes, pass `self.virtual_sequence.producer_responses` and a PDF filename to `render_histogram`. The helper writes the histogram to `sim_build/`. Compare the random and constrained distributions.

```
await self.virtual_sequence.start(seqr=self.tb_env.virtual_sequencer)
render_histogram(
  self.virtual_sequence.producer_responses,
  "sat_tb_around_threshold_test_histogram.pdf",
)
```

- Try increasing the number of sequences and observe how the histogram changes. Are all values equally distributed?
- You can also try and change the constraints and see how the distribution changes.